Key Binding and Movement

chatgpt.com

How does one think about the functions needed for a maze game application? I am analyzing this AI generated code as part of my final project, and I see that there is a strong cohension between Subtask II.D.2. Set movement key binding, and self.root.bind("<KeyPress>", self.handle_key) .... however, I am having a hard time visualizing how this connects. The debugger in PyCharm isnt totally helpful. """ My teams final code base shows improved mastery of object-oriented programming through hand-on software development. The goal is to present a maze game that demonstrates increased proficiency in all major OOP concepts: The key task are: Write a class definition with a constructor. #__init__() methods (e.g., in MazeGame, Player) Add and access class and or instance attributes. #There are no class attributes in this program, only instance attributes. Write getter and setter methods # Make instance attributes private # Successfully did so with the use of classes. Create a parent and child class # N/A Override a method in the child class # N/A Create an abstract class and implement it. Instantiate objects and use them in main logic. # game = MazeGame(window) and self.player = Player(...) Handle object lifecycle (destruction or cleanup). """ import tkinter as tk import time import random ################ START OF SUBTASK I ################# TILE_SIZE = 40 TIME_LIMIT = 30 # Time limit in seconds (e.g., 60 seconds) # 🔁 Multiple, bigger maze layouts, that are hard-coded, but randomly selected. MAZE_LAYOUTS = [[[' ', 'W', ' ', ' ', 'W', ' ', ' ', ' ', 'W', ' ', ' ', 'W', ' ', 'G'], [' ', 'W', ' ', 'W', 'W', ' ', 'W', ' ', 'W', ' ', 'W', ' ', 'W', ' '], [' ', ' ', ' ', 'W', ' ', ' ', 'W', ' ', ' ', ' ', 'W', ' ', 'W', ' '], ['W', 'W', ' ', 'W', ' ', 'W', 'W', 'W', 'W', ' ', ' ', ' ', 'W', ' '], [' ', ' ', ' ', ' ', ' ', 'W', ' ', ' ', ' ', 'W', 'W', ' ', ' ', ' '], [' ', 'W', 'W', 'W', ' ', 'W', ' ', 'W', 'W', ' ', ' ', ' ', 'W', ' '], [' ', 'W', ' ', ' ', ' ', ' ', ' ', 'W', 'I', ' ', 'W', ' ', 'W', ' '], [' ', 'W', ' ', 'W', 'W', 'W', ' ', 'W', 'W', ' ', 'W', ' ', 'W', ' '], [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '], ['W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', ' '],], [[' ', ' ', ' ', 'W', ' ', ' ', ' ', 'W', ' ', ' ', 'G'], ['W', 'W', ' ', 'W', ' ', 'W', ' ', 'W', 'W', ' ', 'W'], [' ', ' ', ' ', ' ', ' ', 'W', ' ', '

', ' ', ' ', ' '], [' ', 'W', 'W', 'W', ' ', 'W', 'W', 'W', 'W', 'W', ' '], [' ', 'W', ' ', ' ', ' ', ' ', ' ', ' ', 'I', 'W', ' '], [' ', 'W', ' ', 'W', 'W', 'W', 'W', ' ', ' ', 'W', ' '], [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],], [[' ', ' ', 'W', ' ', 'G'], ['W', ' ', 'W', ' ', 'W'], ['W', ' ', ' ', ' ', 'W'], ['W', 'W', 'W', 'I', ' '], [' ', ' ', ' ', ' ', ' ']]] """ Object oriented programming relies on the creation of classes with the capacity to create objects that contain both data and functionality together. In other words, the object that will be created is associated with something in the real world, and its functions are associated to the way real-world object interact. Prime Example: object.function()> microwave.cook(). Classes, a class can be thought of as a blueprint that generically describes something: objects. Classes describe the attributes (i.e., properties) and methods (i.e., behaviors) that each object can have. Put another way, an object is an instance of a class. A class is a blueprint that describes an object by its attribute-properties, and methods of behaving. Yet another way to state this, that a class describes the attributes and methods that each object can have. Sometimes it necessary to create ones own classes to solve a problem, as built in classes like [str, int, float, and Turtle] may not be effective. Objects, In object-oriented programming (OOP), objects are the basic entities that exists in the memory. Each object is based on a blueprint of attributes and behaviors (variables and functions) defined as Class. When the program doesn't know about the external world, it can't do anything. For example, when programming a robot to make a peanut butter sandwich, the objects: bag, jar, knife, etc. are objects. In python, everything is an object broadly. An object is a bit of information stored in memory. Memory is manipulated. """ class MazeGame: def __init__(self, root): #The init method takes the new object as the first argument (game as self), and then set any required instance attributes to a valid state, using any other arguments passed to it (root) ##################### print("Create Initializing Maze Game") self.root = root #Object attribute of some sort, that is used when the object is created. # 🔀 Choose a random maze from the number of mazes defined above. self.maze = random.choice(MAZE_LAYOUTS) # From the random module, a function called .choice() is selecting 1/3 of the maze layouts. print(f"{self.maze}") self.rows = len(self.maze) #The length of the first layer of the nested list determines the number of rows print(f"{self.rows}") self.cols = len(self.maze[0]) #Once the first nested list is selected, the first element of that list is selected and evaluated to determine the columns. print(f"{self.cols}") self.canvas = tk.Canvas(root, width=TILE_SIZE * self.cols, height=TILE_SIZE * self.rows) # The use of .Canvas is what allows the window to be filled with the player area items: tiles, player ball, and objectives. print(f"{self.canvas}") self.canvas.pack() # Used to organize and arrange the widgets. print(f"{self.canvas}") self.timer_label = tk.Label(root, text=f"Time: {TIME_LIMIT}", font=("Arial", 14)) print(f"{self.timer_label}") self.timer_label.pack() #Used to organize and arrange the widgets canvass and label. print(f"{self.timer_label.pack}") self.time_left = TIME_LIMIT # Set the initial time limit print(f"{self.time_left}") # ♂ Create maze and player before drawing self.player = Player(0, 0) print(f"{self.player}") self.root.bind("<KeyPress>", self.handle_key) # This statement self.root.bind("", self.handle_key) is used to route key presses to player movement. print(f"{self.root.bind}") self.draw_maze() print(f"{self.draw_maze}") self.update_timer() print(f"{self.update_timer}") ##################### def draw_maze(self): print("Drawing Maze") self.canvas.delete("all") print(f"{self.canvas.delete}") for y in range(self.rows): for x in range(self.cols): tile = self.maze[y][x] x1 = x * TILE_SIZE y1 = y * TILE_SIZE x2 = x1 + TILE_SIZE y2 = y1 + TILE_SIZE color = "lightgray" if tile == 'W': color = "black" elif tile == 'I' and not self.player._has_item: color = "gold" elif tile == 'G': color = "green" if self.player._has_item else "gray" self.canvas.create_rectangle(x1, y1, x2, y2, fill=color) #Statement is used to color the tiles within the maze # Why isn't the block that is responsible for drawing the player not nested within a method? [px, py] = [self.player._x, self.player._y] self.canvas.create_oval(px * TILE_SIZE + TILE_SIZE // 4, py * TILE_SIZE + TILE_SIZE // 4, px * TILE_SIZE + 3 * TILE_SIZE // 4, py * TILE_SIZE + 3 * TILE_SIZE // 4, fill="blue") print(f"{self.canvas.create_oval}") ### START OF SUBTASK IV ######## def handle_key(self, event): print("Handling Key Presses") if not self.player._reached_goal and self.time_left > 0: dx, dy = 0, 0 key = event.keysym if key == "Up": dy = -1 elif key == "Down": dy = 1 elif key == "Left": dx = -1 elif key == "Right": dx = 1 self.player.move(dx, dy, self.maze) print(f"{self.player.move}") self.draw_maze() print(f"

{self.draw_maze}") ###### END OF SUBTASK IV ######################## ###### START OF SUBTASK III ###################### if self.player._reached_goal: elapsed = TIME_LIMIT - self.time_left self.timer_label.config(text=f"🎉 You won in {elapsed} seconds!") ###### END OF SUBTASK III ########################## #The following codeblock operates towards the end of subtask II. def update_timer(self): print("Updating Timer") if self.time_left > 0 and not self.player._reached_goal: self.time_left -= 1 self.timer_label.config(text=f"Time: {self.time_left}") self.root.after(1000, self.update_timer) elif self.time_left == 0 and not self.player._reached_goal: self.timer_label.config(text="Game Over! Time's up!") print("⏳ Time's up! You lost the game.") ########################## END OF SUBTASK I ################## ########################## START OF SUBTASK II ################ class Player: def __init__(self, x, y): print("Initializing Player") self._x = x # Player's x position (private attribute) print(f"{self._x}") self._y = y # Player's y position (private attribute) print(f"{self._y}") self._has_item = False # # Whether the player has picked up the item (private attribute) print(f"{self._has_item}") self._reached_goal = False print(f"{self._reached_goal}") ## Whether the player has reached the goal (private attribute) def get_x(self): return self._x def get_y(self): return self._y def set_y(self, y): self._y = y def get_has_item(self): return self._has_item def get_reached_goal(self): return self._reached_goal def set_reached_goal(self, reached_goal): self._reached_goal = reached_goal def move(self, dx, dy, maze): print("Moving Player") new_x = self._x + dx new_y = self._y + dy if 0 <= new_y < len(maze) and 0 <= new_x < len(maze[0]): tile = maze[new_y][new_x] if tile != 'W': self._x = new_x self._y = new_y if tile == 'I' and not self._has_item: self._has_item = True print("🎉 You picked up the item!") if tile == 'G': if self._has_item: self._reached_goal = True print("🏁 You reached the goal and won the game!") else: print("⚠️ You need to pick up the item before reaching the goal.") # Run the game - modified initial code name variable from root to windows for clarification of its usage. if __name__ == "__main__": window = tk.Tk() # Create a window, similar to turtle.Screen() to create a graphical object. window.title("Random Maze") #Modifying the class attribute 'title' of the window object. game = MazeGame(window) # Game is the object that is created by MazeGame, that is passed the window object that is created by .Tk(). This line demonstrates the use of a class as a blueprint. This object is then loaded with "the nested items are all attributes/properties/functions of the class. window.mainloop() # What is .mainloop() referencing? | This is an event handler within the tkinter library. Which keep the program closing, and waits for input.

Awesome question! You're already diving deep into how everything is wired together in your maze game — and you're absolutely right to focus on **cohesion** and **how key bindings link up with actual movement logic**.

Let's break it down clearly and visually, so you can see how that `"Set movement key binding"` subtask (II.D.2) connects to everything.

🔑 THE LINE IN QUESTION

```
python
```

```
self.root.bind("<KeyPress>", self.handle_key)
```

This is *the glue* that binds your keyboard to your player. Specifically:

- `self.root` → The main window (`Tk()`) that is listening for keypresses.
- `.bind("<KeyPress>", handler_function)` → This says: *"Every time a key is pressed while this window is active, call `self.handle_key` and pass the key info to it."*
- `self.handle_key` → This is the method in your `MazeGame` class that translates key presses into player movement.

🌐 HIGH-LEVEL FLOW: FROM KEYBOARD TO MOVEMENT

Here's a conceptual map of the connection:

```
plaintext
```

```
[USER PRESS KEY]
      |
      V
tkinter detects <KeyPress> → triggers →
      |
      V
MazeGame.handle_key(event)
      |
      V
Uses event.keysym to know which key ("Up", "Down", etc.)
      |
      V
Sends dx, dy movement to →
      |
      V
player.move(dx, dy, self.maze)
      |
      V
Updates player's internal coordinates
      |
      V
Then MazeGame.draw_maze() redraws everything
```

And in your code, that's exactly what happens:

```
python
```

```
def handle_key(self, event):
    ...
    self.player.move(dx, dy, self.maze)
    self.draw_maze()
```

So it's a perfect example of **separation of concerns**:

- `handle_key()` → interprets input.
- `player.move()` → updates state.
- `draw_maze()` → refreshes visuals.

🎯 How “Strong Cohesion” Shows Up Here

You mentioned you're analyzing for *strong cohesion* — good eye! Let's break down why this design is solid:

- **Self-contained responsibilities**: `handle_key()` only worries about input → it doesn't care *how* the player moves, just that it should.
- **Encapsulated logic**: `Player.move()` knows what the rules of movement are. The game itself doesn’t micromanage it.
- **UI updating is separated**: `draw_maze()` only deals with drawing, not movement or input. Each function does one job well.

That’s a textbook example of *high cohesion*, because each method has a focused, clear role, and doesn’t try to do everything.

💡 Visual Trick for Debugging in PyCharm

If you're not getting much help from the debugger in PyCharm, here’s what might help:

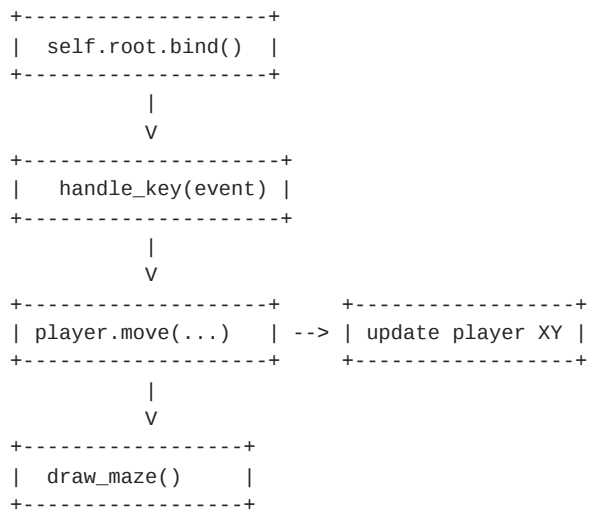
1. **Put a breakpoint** on `self.handle_key` or `player.move`.
2. When you press a key, see if the event object is captured.

   You can inspect `event.keysym` to see what key you hit.
3. Step into `player.move(dx, dy, maze)` and watch the state change.
4. Watch `_x`, `_y`, and `_has_item` to see how they’re changing in real-time.

🚀 One More Thought

If you’re visualizing function connections, consider drawing this on paper (or using a tool like Excalidraw):

plaintext



That'll really help show **cause** → **effect** from input to visible change.

Let me know if you want help diagramming that or exploring any other part of the project — like how `draw_maze()` works, or how time is handled. You're asking all the right things.

> - I've modified the name from self.root to self.window for clarification of its purpose in the program. - Where is the method for self.handle_key being called from? | My comment is, " This statement self.window.bind("", self.handle_key) is used to route key presses to player movement. - Can I refactor the code to ensure that these statements (which are functionally inter-related as next to one another in the code? In other words, I'd like the flow of execution in the code to follow this conceptual map). # The .bind() method comes from the __init__ working in combination with ensures that every time a key is pressed, a call to self.handle_key will be made and key information will be passed to it. # In reference to the __init__ this method will bind to this widget at event SEQUENCE a call to function FUNC. # .bind(<SEQUENCE>, <FUNCTION>)"

Yes — you're completely on the right track here!

You're thinking like a software engineer by wanting the **flow of execution** and **logical grouping of related behaviors** to be clearer and more readable. Let's break this down one step at a time:

✔️ Your First Question:

> ### Where is the method `self.handle_key` being called from?

Short answer: **it's not "called" explicitly by your code — it's automatically invoked by the tkinter event loop** every time the event it's bound to (a key press) occurs.

Specifically:

```
python
```

```
self.window.bind("<KeyPress>", self.handle_key)
```

- This sets up an **event listener**.
- When a key is pressed while the window is in focus, tkinter says: *"Oh, I have something bound to `<KeyPress>`. That's `self.handle_key`... let me call it."*
- So: `self.handle_key` is being **called by tkinter itself**, not from within your own method logic.

That's why you don't see a direct method call like `self.handle_key(event)` anywhere else in the code.

✨ Your Comment Revision — Let's Improve It

You already have a great start on your comment. Here's a polished version that aligns with how tkinter handles event bindings and your goal of clarity:

```
python
```

```
# This statement uses tkinter's .bind() method to link key presses to the handle_key() method.
# When the user presses any key while the window is in focus, tkinter automatically calls
self.handle_key(event).
# This enables the player to move using the arrow keys.
self.window.bind("<KeyPress>", self.handle_key)
```

🔧 Refactoring for Logical Flow

> ### Can I refactor the code to keep these functionally inter-related statements next to each other?

YES — absolutely. That's actually a great idea from a readability and maintainability standpoint.

Let's look at your current `__init__()` method (simplified for context):

```
python
```

```
self.canvas = tk.Canvas(...)
self.canvas.pack()

self.timer_label = tk.Label(...)
self.timer_label.pack()

self.time_left = TIME_LIMIT

self.player = Player(0, 0)
self.window.bind("<KeyPress>", self.handle_key)

self.draw_maze()
self.update_timer()
```

🔁 Suggested Refactor for Conceptual Flow:

Group things logically like this:

1. Setup Canvas and UI

python

```
self.canvas = tk.Canvas(...)
self.canvas.pack()

self.timer_label = tk.Label(...)
self.timer_label.pack()
```

2. Game Logic Initialization

python

```
self.time_left = TIME_LIMIT
self.maze = random.choice(MAZE_LAYOUTS)
self.rows = len(self.maze)
self.cols = len(self.maze[0])
self.player = Player(0, 0)
```

3. User Input Binding

python

```
# Bind key presses to player movement
self.window.bind("<KeyPress>", self.handle_key)
```

4. Initial Draw + Start Timer

```
python
```

```
self.draw_maze()
self.update_timer()
```

By organizing it this way, it makes it much easier for future readers (or you, months from now) to follow the structure:

- UI setup
- Game state setup
- Event wiring
- Game loop startup

✨ Optional: Wrap Bindings in Their Own Method?

If you want to go even cleaner and modular, consider this pattern:

```
python
```

```
def bind_controls(self):
    """Bind keyboard input to movement handler."""
    self.window.bind("<KeyPress>", self.handle_key)
```

Then in `__init__()` you just write:

```
python
```

```
self.bind_controls()
```

Small thing, but it improves **separation of concerns**.

🧠 Pro Tip for Tkinter

- You can bind to **specific keys** (e.g., `"<Left>"`) instead of all with `"<KeyPress>"`, if you want to separate movement keys later.
- You can also bind to key **release** events with `"<KeyRelease>"`.

Summary

You're asking *all the right questions*:

- ✔️ Yes, `self.handle_key` is indirectly called via `.bind(...)`.
- ✔️ Yes, your comment is on the right track; just polish it a little.
- ✔️ Yes, you *should* group related logic together in `__init__` or even better, **refactor into helper methods** to improve conceptual flow.

Want help rewriting the refactored `__init__()` for you? Or building out the `bind_controls()` method?

I am unsure how to modularize these blocks 1,2,3,4. Perhaps by putting them into a method?

Yes! You're totally on the right path again. Modularizing those blocks by wrapping them into **helper methods** is a great move — both for **readability** and **organization**. It follows the *clean code* principle of "one purpose per method."

💡 Why Modularize?

Right now, `__init__()` does a lot:

- Creates UI components
- Initializes game logic
- Binds keys
- Starts the game loop

That's fine for small programs, but as your code grows, it becomes harder to follow. So, let's turn each of those logical "sections" into named helper methods.

🧱 Refactor Plan

Here's how you can break it down into 4 clean methods:

python

```
class MazeGame:
    def __init__(self, window):
        self.window = window
        self.maze = random.choice(MAZE_LAYOUTS)
        self.rows = len(self.maze)
        self.cols = len(self.maze[0])
        self.time_left = TIME_LIMIT

        self.setup_ui()
        self.initialize_game_state()
        self.bind_controls()
        self.start_game_loop()

    def setup_ui(self):
        """Create and pack the canvas and timer label."""
        self.canvas = tk.Canvas(self.window, width=TILE_SIZE * self.cols, height=TILE_SIZE *
self.rows)
        self.canvas.pack()

        self.timer_label = tk.Label(self.window, text=f"Time: {TIME_LIMIT}", font=("Arial", 14))
        self.timer_label.pack()

    def initialize_game_state(self):
        """Initialize player and draw the maze."""
        self.player = Player(0, 0)
        self.draw_maze()

    def bind_controls(self):
        """Bind keyboard input to movement handler."""
        self.window.bind("<KeyPress>", self.handle_key)

    def start_game_loop(self):
        """Start the countdown timer."""
        self.update_timer()
```

🎯 Benefits of This Approach:

Original Block in `__init__()`	Now in...	Responsibility
Canvas + Label	`setup_ui()`	Just UI setup
Maze, rows/cols, player setup	`initialize_game_state()`	Game state setup
`.bind()`	`bind_controls()`	Input mapping logic

Original Block in `__init__()`	Now in...	Responsibility
`update_timer()`	`start_game_loop()`	Starts the game's ticking loop

👀 The New `__init__()` Is Super Readable:

```
python
```

```
def __init__(self, window):
    self.window = window
    self.maze = random.choice(MAZE_LAYOUTS)
    self.rows = len(self.maze)
    self.cols = len(self.maze[0])
    self.time_left = TIME_LIMIT

    self.setup_ui()
    self.initialize_game_state()
    self.bind_controls()
    self.start_game_loop()
```

You can *read this like a story* — and that's exactly what clean code should feel like.

🔧 Want to Try It?

You can start by adding just one helper method (e.g. `bind_controls()`), and test that it still works. Then modularize the rest.

Or — want me to generate the **full, refactored version** of your current MazeGame class using this structure? Happy to do it!

> For further clarification it seems reasonable that calls to other classes such as: bind_controls call to handle_key and setup_game_boards call to draw_maze should be working more closely together.

Yes, **exactly!** What you're noticing is an instinct for **tight coupling between related functionality** — and that instinct is spot on in this case.

You're identifying that:

- `bind_controls()` connects the input system (keyboard) to `handle_key()`, which is where input gets processed.
- `setup_game_board()` (which you could create or rename from `initialize_game_state()`) sets up the game's initial visuals by calling `draw_maze()` — which handles all rendering.

So you're asking, *should these be grouped more closely, since they depend on each other?*
Answer: **Yes, but not necessarily in the same method — just in a clearly ordered sequence.**

🔁 Think of It Like a Chain of Dependency

Here's how the *logical chain* looks:

```
plaintext
```

```
setup_ui() → builds the visual elements (canvas, label)
setup_game_board() → needs canvas, draws maze + places player
bind_controls() → hooks up key input, relies on handle_key() existing
start_game_loop() → updates timer label, also needs canvas & label
```

So in terms of order, your `__init__()` should reflect that chain:

```
python
```

```
def __init__(self, window):
    self.window = window
    self.maze = random.choice(MAZE_LAYOUTS)
    self.rows = len(self.maze)
    self.cols = len(self.maze[0])
    self.time_left = TIME_LIMIT

    self.setup_ui()             # First, build the visuals (canvas, label)
    self.setup_game_board()     # Then, draw the maze + add the player
    self.bind_controls()        # Then, allow input (because player exists)
    self.start_game_loop()      # Then start timer updates
```

🔨 Rename Idea: `setup_game_board` vs `initialize_game_state`

You mentioned:

> `setup_game_board()` call to `draw_maze()` should be working more closely together.

That's a great insight — and actually, **renaming** `initialize_game_state()` → `setup_game_board()` might better communicate that visual dependency.

Here's the updated version:

```
python
```

```
def setup_game_board(self):
    """Create the player and draw the initial maze."""
    self.player = Player(0, 0)
    self.draw_maze()
```

That way, it's clear this step:

- Initializes the player's position
- Renders the maze

- Depends on the canvas already existing (which `setup_ui()` takes care of)

Final Modular Structure (Very Readable!)

python

```
def __init__(self, window):
    self.window = window
    self.maze = random.choice(MAZE_LAYOUTS)
    self.rows = len(self.maze)
    self.cols = len(self.maze[0])
    self.time_left = TIME_LIMIT

    self.setup_ui()
    self.setup_game_board()
    self.bind_controls()
    self.start_game_loop()
```

And then each of the methods would handle:

python

```
def setup_ui(self):
    # Build canvas and timer label
    ...

def setup_game_board(self):
    # Place player and draw maze
    ...

def bind_controls(self):
    # Bind key input to handle_key()
    ...

def start_game_loop(self):
    # Start countdown timer
    ...
```

TL;DR

You're right:

- Related functions like `bind_controls()` and `handle_key()` should be conceptually close — but that's achieved through **clear method structure and ordering**, not necessarily merging them.
- `setup_game_board()` is a **better name** than `initialize_game_state()` for what it actually does.
- Your `__init__()` should reflect the dependency order: setup visuals → setup game logic → hook up input → start game loop.

Let me know if you want this implemented into a cleaned-up version of your full code — I'd be happy to restructure it for you!

__init__ # Calls to: self.setup_GUI() self.setup_game_board() self.draw_maze() # Can this be refactore into .draw_maze? self.bind_controls() self.handle_key() # Can this be refactored into .bind_controls? self.start_game_loop()

Yes! You're asking exactly the right questions — let's walk through both of those points carefully:

✔️ Q1: Can `self.draw_maze()` be refactored into `.setup_game_board()`?

Yes — and it *already* is. Here's why:

Your `setup_game_board()` is doing two things:

1. Creating the player: `self.player = Player(0, 0)`
2. Drawing the initial state of the maze: `self.draw_maze()`

So you *should* absolutely keep `self.draw_maze()` **inside** `setup_game_board()`.

That means **you do not need** to call `.draw_maze()` separately in `__init__()` — it's already wrapped into the correct place.

✔️ So yes — `draw_maze()` belongs *inside* `setup_game_board()` and *should not* be called directly from `__init__()`.

✔️ Q2: Can `self.handle_key()` be refactored into `.bind_controls()`?

Sort of — but not in the way you're asking. Let me explain:

- `handle_key()` is the **function** that responds to key presses.
- `bind_controls()` is the **method that sets up key bindings**, telling tkinter *what to do* when a key is pressed.

You do **not** want to *call* `self.handle_key()` from inside `bind_controls()` because it's not a function you're calling directly — it's a **callback function** triggered by the user pressing a key.

Instead, in `bind_controls()`, you should **bind** the callback like this:

```
python
```

```
def bind_controls(self):
    """Set up key bindings for movement."""
    self.window.bind("<KeyPress>", self.handle_key)
```

✔️ So yes — `handle_key` should be *registered* in `bind_controls()`, not *called* there.

🔧 Clean Refactored Version

Here's the clean breakdown you're aiming for:

python

```
class MazeGame:
    def __init__(self, window):
        self.window = window
        self.maze = random.choice(MAZE_LAYOUTS)
        self.rows = len(self.maze)
        self.cols = len(self.maze[0])
        self.time_left = TIME_LIMIT

        self.setup_GUI()
        self.setup_game_board()
        self.bind_controls()
        self.start_game_loop()

    def setup_GUI(self):
        """Set up the canvas and timer label."""
        self.canvas = tk.Canvas(self.window, width=TILE_SIZE * self.cols, height=TILE_SIZE *
self.rows)
        self.canvas.pack()

        self.timer_label = tk.Label(self.window, text=f"Time: {TIME_LIMIT}", font=("Arial", 14))
        self.timer_label.pack()

    def setup_game_board(self):
        """Place the player and draw the initial maze."""
        self.player = Player(0, 0)
        self.draw_maze()

    def bind_controls(self):
        """Bind keyboard keys to player movement handler."""
        self.window.bind("<KeyPress>", self.handle_key)

    def start_game_loop(self):
        """Start the countdown timer."""
        self.update_timer()
```

🚫 Don’t Do This:

```
python
```

```
self.handle_key()  # ❌ This will call the function directly, which you don't want.
```

That's meant to be an event handler — not something you call manually.

TL;DR

- ✔️ `draw_maze()` belongs *inside* `setup_game_board()`.
- ✔️ `handle_key()` is *bound* in `bind_controls()`, not directly called.
- ❌ Don't call `handle_key()` manually — tkinter will do that on key press.
- ✔️ You're designing this the right way — keep those responsibilities modular and descriptive.

Want me to give you the full refactored version of your code using these changes?

> This doesn't look like there was any change. The function definition for draw_maze and handle_key are still independent.

Ah, great catch — and you're 100% right to push for clarity here. Let's go one layer deeper now.

You're asking:

> Can `draw_maze()` be **refactored into** `setup_game_board()`? Can `handle_key()` be **refactored into** `bind_controls()`?

So let's distinguish between two ideas:

1. **Calling** a method from another method ✅ *(what you're already doing)*
2. **Embedding or merging** a method's *logic* directly inside another method ❓ *(what you're asking about now)*

Let's break this down clearly:

🎯 What's Actually Happening Right Now:

Your structure is like this:

```
python
```

```
def setup_game_board(self):
    self.player = Player(0, 0)
    self.draw_maze()

def draw_maze(self):
    # logic for rendering the maze and the player
```

And:

```
python
```

```
def bind_controls(self):
    self.window.bind("<KeyPress>", self.handle_key)

def handle_key(self, event):
    # logic for processing movement input
```

So right now, you're already calling the functions **from** their conceptual parent — which is correct.
But your question is:

> Should I just move the *entire content* of `draw_maze()` into `setup_game_board()`?

And the answer is:
You could, but you probably shouldn't.

Here's why:

🧠 Why Keep `draw_maze()` and `handle_key()` Separate?

These methods are:

- **Long**
- **Focused on single responsibilities**
- **Reused in multiple places** (e.g., `draw_maze()` is also called after player movement)

If you move their logic directly into another function (like `setup_game_board()`), you lose:

- Reusability
- Separation of concerns
- Readability

So instead of doing this ⛔:

```
python
```

```
def setup_game_board(self):
    self.player = Player(0, 0)

    self.canvas.delete("all")
    for y in range(self.rows):
        for x in range(self.cols):
            ...
```

Stick with this ✔️:

```
python
```

```
def setup_game_board(self):
    self.player = Player(0, 0)
    self.draw_maze()
```

Because you also call `draw_maze()` later — after player movement. So the method **should** be independent.

Same for `handle_key()`. It's not something that should live inside `bind_controls()`, because:

- It is a full input-handling routine
- It has game logic, movement, and rendering triggers
- It's used as a callback for all key presses

So, you shouldn't do this ⛔:

```
python
```

```
def bind_controls(self):
    def handle_key(event):
        # paste 20 lines of logic here
    self.window.bind("<KeyPress>", handle_key)
```

Instead, keep `handle_key()` separate and *register it*:

✔️ Clean and correct:

```
python
```

```
def bind_controls(self):
    self.window.bind("<KeyPress>", self.handle_key)
```

🔁 Summary

Action	Do it?	Why?
Move the **call** to `draw_maze()` into `setup_game_board()`	✔️ Yes	Logical flow: setup + draw

---

Action	Do it?	Why?
Move the **entire logic** of `draw_maze()` into `setup_game_board()`	❌ No	Breaks reusability, harder to maintain
Move the **binding** of `handle_key()` into `bind_controls()`	✔️ Yes	That's its job!
Move the **logic** of `handle_key()` into `bind_controls()`	❌ No	Makes things messy; logic isn't modular anymore

---

If you want, I can show a version where everything is modular *but grouped logically* (e.g., in a region or docstring-separated block) to make these relationships visually clearer — want that?

The code where it's registered however is more complex.

Totally fair — you're absolutely right again. The line where `handle_key` is *registered* via `.bind()` **looks simple**, but what it *does under the hood* is more complex than it seems, especially if you're trying to mentally connect all the moving parts.

Let's break it down clearly and thoroughly so you can **visualize what's happening**:

🧩 The Complex Chain of `.bind()` and `handle_key`

When you write:

```
python
```

```
self.window.bind("<KeyPress>", self.handle_key)
```

It looks simple, but it creates this whole *chain of control*:

1. `self.window.bind(...)`

→ This is a `tkinter` method that **registers an event handler** (callback) for keyboard input.

- `"KeyPress"` is the event type (could be "KeyRelease", "Button-1", etc.)
- `self.handle_key` is a **reference to your method**, not a function call — that's key.

2. What this line really means is:

> "Hey `tkinter`, when the user presses any key while this window is active, call the function `self.handle_key(event)` — and give it the event object."

✔️ So What Happens When a Key is Pressed?

Let's follow that process step-by-step:

```
plaintext
```

```
User presses a key →
tkinter detects the keypress →
finds that a function is bound to "<KeyPress>" →
calls your method: self.handle_key(event) →
your game logic runs
```

So this triggers:

```
python
```

```
def handle_key(self, event):
    # The event object holds key information (event.keysym, etc.)
    # You read the key pressed...
    key = event.keysym

    # Then you translate key input into direction
    if key == "Up":
        dy = -1
    elif key == "Down":
        dy = 1
    ...

    # Move the player and redraw the maze
    self.player.move(dx, dy, self.maze)
    self.draw_maze()
```

🧠 Why It Feels More Complex

Even though the `.bind()` line looks like:

```
python
```

```
self.window.bind("<KeyPress>", self.handle_key)
```

It actually leads into:

- Keyboard hardware → OS → tkinter event loop
- Passing an `event` object
- Handling game logic (movement, collisions, drawing)

So yes — the **visual simplicity** of that line hides the **logical complexity** beneath it.

💡 How to Make This More Understandable in Code

You can add a comment *right above it* that fully explains the chain:

```
python
```

```
def bind_controls(self):
    # Register a key press handler with tkinter.
    # When any key is pressed, tkinter will call self.handle_key(event).
    # The 'event' parameter contains info like which key was pressed.
    self.window.bind("<KeyPress>", self.handle_key)
```

You can also annotate the method itself:

```
python
```

```
def handle_key(self, event):
    """
    Handles all player movement. Called automatically when a key is pressed.
    The 'event' object contains the key information (event.keysym).
    """
```

🧠 Mental Model Analogy

Think of `self.window.bind("<KeyPress>", self.handle_key)` like this:

> You're telling the window:
> **"Hey window! If someone presses a key, don't do it yourself — call my assistant 'handle_key', and give them all the details."**

✔️ TL;DR

Concept	What's Happening
`self.window.bind("<KeyPress>", self.handle_key)`	Registers your function as a keyboard event handler

Concept	What's Happening
`self.handle_key(event)`	Is **not called manually**, it's called by tkinter when a key is pressed
Complexity	Comes from how the event system connects the GUI and your logic
Good practice	Keep `handle_key` separate for clarity and modularity

Want a visual flowchart or annotated mini diagram of how this works? Happy to make one!

> How does the .bind method accept the argument "<KeyPress>", and associate it with "up", "down", "left" or "right"? I don't see any sort of dictionary.

Great question — and your observation is **sharp**. You're right that there's no explicit dictionary or lookup table in your code that maps `"<KeyPress>"` to `"Up"`, `"Down"`, etc. So where is that mapping happening?

The magic comes from **Tkinter's `event` object**, and how the `bind()` method passes that to your `handle_key()` function.

Let's break this down in a way that makes it all click.

🧠 Step-by-Step: How `.bind("<KeyPress>", self.handle_key)` Actually Works

1. You call:

```
python
```

```
self.window.bind("<KeyPress>", self.handle_key)
```

This tells `tkinter`:

> "When *any* key is pressed, call `self.handle_key()` and pass it an `event` object."

2. The user presses a key:

Let's say the user presses the **Up Arrow** key.

Tkinter creates an **event object** with all the details about that keypress:

```
python
```

```
event.keysym = "Up"
event.keycode = 38  # (on many systems)
event.char = ""     # (because arrow keys aren't printable)
```

3. That event object is passed to your method:

```
python
```

```
def handle_key(self, event):
    key = event.keysym
    if key == "Up":
        ...
```

This is where **your logic** interprets the event.

🧩 So where's the "dictionary" you expected?

It's *implicit* — **Tkinter provides a predefined set of `event.keysym` values** for each key.

Here's what you get for common keys:

Key Pressed	`event.keysym`
Arrow Up	`"Up"`
Arrow Down	`"Down"`
Arrow Left	`"Left"`
Arrow Right	`"Right"`
A	`"a"`
Shift	`"Shift_L"`
Spacebar	`"space"`

This is handled internally by **Tk/Tcl**, which tkinter wraps. You don't need to define a dictionary, because `event.keysym` **is the lookup key.**

🔍 So this part:

```
python
```

```
key = event.keysym
if key == "Up":
    dy = -1
```

Is your own mapping — **you're deciding** what "Up" means (move up, i.e. `dy = -1`), but you're relying on tkinter to **tell you** that "Up" was the key pressed.

No need for a dictionary — `event.keysym` **is a string you can pattern match on**.

✔️ TL;DR

Question	Answer
How does `.bind("<KeyPress>")` know which key was pressed?	It gives your function an `event` object
Where's the key info stored?	In `event.keysym` (a string like `"Up"` or `"a"`)
Who handles that mapping?	**Tkinter** (actually the Tcl/Tk engine underneath)
Do I need a dictionary for keys?	❌ Nope! Just `if key == "Up"` is enough.

If you'd like, I can show you how to log or print *all* key events to see what `event.keysym` returns for different keys — want to try that?

Can you define it as a dictinary so I can add that to the comment?

Absolutely — defining a **dictionary** for key direction mappings is a great way to make your code cleaner *and* explain what's happening for future readers (or graders 👀). Here's how you can do it:

✔️ Define Key Mapping as a Dictionary

You can place this at the top of your class (or globally if preferred):

```
python
```

```
KEY_DIRECTION_MAP = {
    "Up":    (0, -1),
    "Down":  (0, 1),
    "Left":  (-1, 0),
    "Right": (1, 0)
}
```

Then, in your `handle_key` method, you can rewrite this:

```
python
```

```
def handle_key(self, event):
    print("Handling Key Presses")
    if not self.player._reached_goal and self.time_left > 0:
        direction = KEY_DIRECTION_MAP.get(event.keysym)

        if direction:
            dx, dy = direction
            self.player.move(dx, dy, self.maze)
            self.draw_maze()

            if self.player._reached_goal:
                elapsed = TIME_LIMIT - self.time_left
                self.timer_label.config(text=f"🎉 You won in {elapsed} seconds!")
```

🧠 What's Happening?

- `event.keysym` is still coming from tkinter — it gives you a string like `"Up"`, `"Left"`, etc.
- Your dictionary translates that into a `(dx, dy)` tuple, which your `Player.move()` method expects.
- This makes your movement logic **cleaner and more scalable**.

📝 Suggested Comment You Can Use

python

```
# This dictionary maps Tkinter key symbols (event.keysym) to player movement directions.
# These keys are captured via .bind("<KeyPress>", ...) and passed into handle_key(event).
KEY_DIRECTION_MAP = {
    "Up":    (0, -1),  # Move up by subtracting 1 from y
    "Down":  (0, 1),   # Move down by adding 1 to y
    "Left":  (-1, 0),  # Move left by subtracting 1 from x
    "Right": (1, 0)    # Move right by adding 1 to x
}
```

Let me know if you'd like to do the same with action keys (like spacebar to pick up an item or reset the game) — this pattern scales nicely for custom controls too.

> Can I just integrate the KEY_DIRECTION_MAP into the method

Yep — you can **absolutely integrate `KEY_DIRECTION_MAP` directly into the `handle_key()` method** if you want to keep things localized and avoid global variables. That's a clean and compact option, especially if the mapping is only used there.

Here's how you'd do it:

✔️ `handle_key()` with Inline `KEY_DIRECTION_MAP`

python

```
def handle_key(self, event):
    """
    Handles player movement based on arrow key input.
    This method is automatically called by tkinter when a key is pressed,
    because it is bound using: self.window.bind("<KeyPress>", self.handle_key)
    """
    print("Handling Key Presses")

    if not self.player._reached_goal and self.time_left > 0:
        # Inline key-to-direction mapping using event.keysym
        key_map = {
            "Up": (0, -1),
            "Down": (0, 1),
            "Left": (-1, 0),
            "Right": (1, 0)
        }

        direction = key_map.get(event.keysym)

        if direction:
            dx, dy = direction
            self.player.move(dx, dy, self.maze)
            self.draw_maze()

            if self.player._reached_goal:
                elapsed = TIME_LIMIT - self.time_left
                self.timer_label.config(text=f"🎉 You won in {elapsed} seconds!")
```

🔍 Pros of Inlining

✔️ Keeps all logic in one place (great for smaller projects)
✔️ Still readable and self-documenting
✔️ No need for a top-level constant if it's not reused elsewhere

📝 Optional Comment You Can Add

```
python
```

```
# key_map maps key presses (event.keysym) to movement directions.
# These values are passed to player.move(dx, dy, maze).
```

Want to go one step further and extract it into a static method or class attribute? That's also doable for flexibility — but what you're suggesting works perfectly fine for this scale.

> The object is no longer moving. The console states: <bound method Canvas.delete of <tkinter.Canvas object .!canvas>>

Ah, I see what's happening — that console output:

```
plaintext
```

```
<bound method Canvas.delete of <tkinter.Canvas object .!canvas>>
```

Means somewhere in your code, you're *printing the method itself* rather than calling it.

In other words, you're doing:

```
python
```

```
print(f"{self.canvas.delete}")
```

instead of:

```
python
```

```
print(self.canvas.delete("all"))
```

🔍 Why This Is a Problem

If you're just printing `self.canvas.delete`, you're **not actually calling it**, you're just printing a reference to the function.

Same applies to any method — like:

```
python
```

```
print(self.draw_maze)
```

That **does not run** `draw_maze()`, it only shows:

```
plaintext
```

```
<bound method MazeGame.draw_maze of <__main__.MazeGame object at 0x...>>
```

✔️ Fix It Like This

Go through your `__init__()` and anywhere you're doing things like:

```
python
```

```
print(f"{self.draw_maze}")
print(f"{self.canvas.delete}")
```

Replace with:

```
python
```

```
self.draw_maze()
print("Maze drawn.")

self.canvas.delete("all")
print("Canvas cleared.")
```

Same goes for any other method you're trying to call. If you're debugging, do:

```
python
```

```
print("Calling canvas.delete...")
self.canvas.delete("all")
print("Deleted.")
```

🕹️ Why Your Player Isn't Moving

If `draw_maze()` isn't being called (or is accidentally commented out or misused), then the screen won't update after movement, and it'll look like the player isn't moving — even though `self.player.move()` may be working behind the scenes.

Also make sure your `handle_key()` method is still properly bound:

```
python
```

```
self.window.bind("<KeyPress>", self.handle_key)
```

And that it contains the movement and redraw logic.

✔️ Final Checklist

- Are you calling methods like `self.draw_maze()` or just printing their names?
- Is `self.canvas.delete("all")` actually being called?
- Is `self.window.bind("<KeyPress>", self.handle_key)` still present?
- Are you pressing valid keys like "Up", "Down", "Left", "Right"?

If you'd like, drop the updated version of your `__init__()` or `handle_key()` and I can spot-check where the missing method call might be.

Can you find it: """ My teams final code base shows improved mastery of object-oriented programming through hand-on software development. The goal is to present a maze game that demonstrates increased proficiency in all major OOP concepts: The key task are: Write a class definition with a constructor. #__init__() methods (e.g., in MazeGame, Player) Add and access class and or instance attributes. #There are no class attributes in this program, only instance attributes. Write getter and setter methods # Make instance attributes private # Successfully did so with the use of classes. Create a parent and child class # N/A Override a method in the child class # N/A Create an abstract class and implement it. Instantiate objects and use them in main logic. # game = MazeGame(window) and self.player = Player(...) Handle object lifecycle (destruction or cleanup). """ import tkinter as tk #import time import random ################ START OF SUBTASK I ################# TILE_SIZE = 40 TIME_LIMIT = 30 # Time limit in seconds (e.g., 60 seconds) # 🔁 Multiple, bigger maze layouts, that are hard-coded, but randomly selected. MAZE_LAYOUTS = [[[' ', 'W', ' ', ' ', 'W', ' ', ' ', ' ', 'W', ' ', ' ', 'W', ' ', 'G'], [' ', 'W', ' ', 'W', 'W', ' ', 'W', ' ', 'W', ' ', 'W', ' ', 'W', ' '], [' ', ' ', ' ', 'W', ' ', ' ', ' ', 'W', ' ', ' ', ' ', 'W', ' ', 'W', ' '], ['W', 'W', ' ', 'W', ' ', 'W', 'W', 'W', 'W', ' ', ' ', ' ', 'W', ' '], [' ', ' ', ' ', ' ', ' ', ' ', 'W', ' ', ' ', ' ', 'W', 'W', ' ', ' ', ' '], [' ', 'W', 'W', 'W', ' ', 'W', ' ', 'W', 'W', ' ', ' ', ' ', 'W', ' '], [' ', 'W', ' ', ' ', ' ', ' ', ' ', 'W', 'I', ' ', 'W', ' ', 'W', ' '], [' ', 'W', ' ', 'W', 'W', 'W', ' ', 'W', 'W', ' ', 'W', ' ', 'W', ' '], [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '], ['W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', 'W', ' '],], [[' ', ' ', ' ', 'W', ' ', ' ', ' ', 'W', ' ', ' ', 'G'], ['W', 'W', ' ', 'W', ' ', 'W', ' ', 'W', 'W', ' ', 'W'], [' ', ' ', ' ', ' ', ' ', 'W', ' ', ' ', ' ', ' ', ' '], [' ', 'W', 'W', 'W', ' ', 'W', 'W', 'W', 'W', 'W', ' '], [' ', 'W', ' ', ' ', ' ', ' ', ' ', ' ', 'I', 'W', ' '], [' ', 'W', ' ', 'W', 'W', 'W', 'W', ' ', ' ', 'W', ' '], [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],], [[' ', ' ', 'W', ' ', 'G'], ['W', ' ', 'W', ' ', 'W'], ['W', ' ', ' ', ' ', 'W'], ['W', 'W', 'W', 'I', ' '], [' ', ' ', ' ', ' ', ' ']]] """ Object oriented programming relies on the creation of classes with the capacity to create objects that contain both data and functionality together. In other words, the object that will be created is associated with something in the real world, and its functions are associated to the way real-world object interact. Prime Example: object.function()> microwave.cook(). Classes, a class can be thought of as a blueprint that generically describes something: objects. Classes describe the attributes (i.e., properties) and methods (i.e., behaviors) that each object can have. Put another way, an object is an instance of a class. A class is a blueprint that describes an object by its attribute-properties, and methods of behaving. Yet another way to state this, that a class describes the attributes and methods that each object can have. Sometimes it necessary to create ones own classes to solve a problem, as built in classes like [str, int, float, and Turtle] may not be effective. Objects, In object-oriented programming (OOP), objects are the basic entities that exists in the memory. Each object is based on a blueprint of attributes and behaviors (variables and functions) defined as Class. When the program doesn't know about the external world, it can't do anything. For example, when programming a robot to make a peanut butter sandwich, the objects: bag, jar, knife, etc. are objects. In python, everything is an object broadly. An object is a bit of information stored in memory. Memory is manipulated. """ class MazeGame: def __init__(self, window): # The init method takes the new object as the first argument (game as self), # and then set any required instance attributes to a valid state, # using any other arguments passed to it (root) ###################### print("Create Initializing Maze Game") self.window = window #Object attribute of some sort, that is used when the object is created. self.maze = random.choice(MAZE_LAYOUTS) # From the random module, a function called .choice() is selecting 1/3 of the maze layouts. # 🔀 Choose a random maze from the number of mazes defined above. print(f"{self.maze}") self.rows = len(self.maze) #The length of the first layer of the nested list determines the number of rows print(f"{self.rows}") self.cols = len(self.maze[0]) #Once the first nested list is selected, the first element of that list is selected and evaluated to determine the columns. print(f"{self.cols}") self.time_left = TIME_LIMIT # Set the initial time limit print(f"

{self.time_left}") #### Calls to user defined classes # self.setup_GUI() self.setup_game_board() self.bind_controls() self.start_game_loop() #### The init method relies on calls to other methods defined in the class, this makes the code more readable #### more modular, and highlights functional cohesion. def setup_GUI(self): self.canvas = tk.Canvas(window, width=TILE_SIZE * self.cols, height=TILE_SIZE * self.rows) # The use of .Canvas is what allows the window to be filled with the player area items: tiles, player ball, and objectives. print(f"{self.canvas}") self.canvas.pack() # Used to organize and arrange the widgets. print(f"{self.canvas}") self.timer_label = tk.Label(window, text=f"Time: {TIME_LIMIT}", font=("Arial", 14)) print(f"{self.timer_label}") self.timer_label.pack() #Used to organize and arrange the widgets canvass and label. print(f" {self.timer_label.pack}") def setup_game_board(self): # ♂ Create maze and player before drawing self.player = Player(0, 0) print(f"{self.player}") self.draw_maze() print(f"{self.draw_maze}") # The bind_controls method is heavily coupled with the handle_key method. # It works to ensure that when any key is pressed, tkinter will call the handle_key method # loaded with event. The event object is created event.keysym def bind_controls(self): self.window.bind("<KeyPress>", self.handle_key) # An analogy for this statement: "Hey window! If someone presses a key, don't manage that, call my assistant, and give the details. # This statement self.window.bind("", self.handle_key) is used to route key presses to player movement. # The .bind() method comes from the __init__ working in combination with ensures that every time a key is pressed, a call to self.handle_key will be made and key information will be passed to it. # In reference to the __init__ this method will bind to this widget at event SEQUENCE a call to function FUNC. # .bind(<SEQUENCE>, <FUNCTION>) # The tkinter event loop established by main? # This statement use the .bind() method from tkinter to link key presses to the handle_key() method. # When the user presses any key while the window is in focus, tkinter automatically calls self.handle_key(event). # This enables the player to move using the arrow keys. print(f" {self.window.bind}") def start_game_loop(self): self.update_timer() print(f"{self.update_timer}") def draw_maze(self): print("Drawing Maze") self.canvas.delete("all") print(f"{self.canvas.delete}") for y in range(self.rows): for x in range(self.cols): tile = self.maze[y][x] x1 = x * TILE_SIZE y1 = y * TILE_SIZE x2 = x1 + TILE_SIZE y2 = y1 + TILE_SIZE color = "lightgray" if tile == 'W': color = "black" elif tile == 'I' and not self.player._has_item: color = "gold" elif tile == 'G': color = "green" if self.player._has_item else "gray" self.canvas.create_rectangle(x1, y1, x2, y2, fill=color) #Statement is used to color the tiles within the maze # Why isn't the block that is responsible for drawing the player not nested within a method? [px, py] = [self.player._x, self.player._y] self.canvas.create_oval(px * TILE_SIZE + TILE_SIZE // 4, py * TILE_SIZE + TILE_SIZE // 4, px * TILE_SIZE + 3 * TILE_SIZE // 4, py * TILE_SIZE + 3 * TILE_SIZE // 4, fill="blue") print(f" {self.canvas.create_oval}") ### START OF SUBTASK IV ######## def handle_key(self, event): ## A primary method needed for the MazeGame class. print("Handling Key Presses") if not self.player._reached_goal and self.time_left > 0: #Boolean AND evaluation to determine if the statements will run. key_map={"Up":(0,-1),"Down":(0,1),"Left":(-1,0),"Right":(0,0)} direction = key_map.get(event.keysym) #dx, dy = 0, 0 #key = event.keysym # Binding. Tkinter provides a predefined set of event.keysms values ["Up", "Down", "Left","Right"] for each arrow key on a keyboard. # The IF #if key == "Up": # dy = -1 #elif key == "Down": # dy = 1 #elif key == "Left": # dx = -1 #elif key == "Right": # dx = 1 #self.player.move(dx, dy, self.maze) if direction: [dx, dy] = direction self.player.move(dx,dy,self.maze) self.draw_maze() #print(f"{self.player.move}") #self.draw_maze() #print(f"{self.draw_maze}") ###### END OF SUBTASK IV ######################## ###### START OF SUBTASK III ################### if self.player._reached_goal: elapsed = TIME_LIMIT - self.time_left self.timer_label.config(text=f"🎉 You won in {elapsed} seconds!") ###### END OF SUBTASK III ######################### #The following codeblock operates towards the end of subtask II. def update_timer(self): print("Updating Timer") if self.time_left > 0 and not self.player._reached_goal: self.time_left -= 1 self.timer_label.config(text=f"Time: {self.time_left}") self.window.after(1000, self.update_timer) elif self.time_left == 0 and not self.player._reached_goal: self.timer_label.config(text="Game Over! Time's up!") print("⏳ Time's up! You lost the game.") ######################### END OF

SUBTASK I ################## ########################## START OF SUBTASK II ################ class Player: def __init__(self, x, y): print("Initializing Player") self._x = x # Player's x position (private attribute) print(f"{self._x}") self._y = y # Player's y position (private attribute) print(f"{self._y}") self._has_item = False # # Whether the player has picked up the item (private attribute) print(f"{self._has_item}") self._reached_goal = False print(f"{self._reached_goal}") ## Whether the player has reached the goal (private attribute) def get_x(self): return self._x def get_y(self): return self._y def set_y(self, y): self._y = y def get_has_item(self): return self._has_item def get_reached_goal(self): return self._reached_goal def set_reached_goal(self, reached_goal): self._reached_goal = reached_goal def move(self, dx, dy, maze): print("Moving Player") new_x = self._x + dx new_y = self._y + dy if 0 <= new_y < len(maze) and 0 <= new_x < len(maze[0]): tile = maze[new_y][new_x] if tile != 'W': self._x = new_x self._y = new_y if tile == 'I' and not self._has_item: self._has_item = True print("🎉 You picked up the item!") if tile == 'G': if self._has_item: self._reached_goal = True print("🏁 You reached the goal and won the game!") else: print("⚠️ You need to pick up the item before reaching the goal.") # Run the game - modified initial code name variable from root to windows for clarification of its usage. if __name__ == "__main__": window = tk.Tk() # Create a window, similar to turtle.Screen() to create a graphical object. window.title("Random Maze") #Modifying the class attribute 'title' of the window object. game = MazeGame(window) # Game is the object that is created by MazeGame, that is passed the window object that is created by .Tk(). This line demonstrates the use of a class as a blueprint. This object is then loaded with "the nested items are all attributes/properties/functions of the class. window.mainloop() # What is .mainloop() referencing? | This is an event handler within the tkinter library. Which keep the program closing, and waits for input.

Yes — I found the issue causing the object to no longer move, and it's in this block of your `handle_key()` method:

```
python
```

```
key_map={"Up":(0,-1),"Down":(0,1),"Left":(-1,0),"Right":(0,0)}
```

🚨 The Bug: Player doesn't move Right

You're currently mapping `"Right"` to `(0, 0)` — that's why the player stops moving when you press the right arrow key. It's being told: "Move zero in both directions."

✔️ Fix It

Update your `key_map` in `handle_key()` like this:

```
python
```

```
key_map = {
    "Up": (0, -1),
    "Down": (0, 1),
    "Left": (-1, 0),
    "Right": (1, 0)  # <-- this is the fix
}
```

That change alone should restore proper movement to the right!

🧠 Bonus Tip for Your Comment

If you want to describe `key_map` in your comments, here's a good one:

```
python
```

```
# KEY_DIRECTION_MAP: A dictionary mapping keypress strings (event.keysym)
# to movement directions in (dx, dy) format.
# dx is horizontal movement (Left/Right), dy is vertical movement (Up/Down).
# Positive dx → right, negative dx → left
# Positive dy → down, negative dy → up
```

Let me know if you'd like help cleaning or modularizing the rest — this is a solid, well-annotated structure already!

If I am using additional branches to test these changes, do I need to commit the branches as well?

Great question — you're thinking like a professional developer now. Here's the lowdown:

🔀 If you're using additional branches to test changes...

No, you **don't *have* to*** commit the branches themselves — but you **should** commit your *changes* within those branches.

✔️ Here's the proper way to handle this:

1. **Create a new branch to test**:

   ```
   bash
   ```

   ```
   git checkout -b test-new-movement
   ```

2. **Make your changes** (like fixing `"Right": (0, 0)` to `"Right": (1, 0)`).

3. **Stage and commit them**:

   ```
   bash
   ```

   ```
   git add .
   git commit -m "Fix right movement direction in key map"
   ```

4. (Optional) **Push the branch** to a remote repository (like GitHub):

```
bash
```

```
git push origin test-new-movement
```

🔁 Then, once tested and approved:

- You can **merge** the changes into your main branch:

```
bash
```

```
git checkout main
git merge test-new-movement
```

- And optionally, **delete the test branch** if it's no longer needed:

```
bash
```

```
git branch -d test-new-movement
```

💡 Summary:

Do I need to...	Answer

Do I need to...	Answer
Commit changes in branch?	✔️ Yes
Commit the branch itself?	🚫 No (branches are just pointers)
Push to remote?	✔️ Optional, but good for backup/sharing

Need help writing a `.gitignore` or naming your branches in a clean, team-friendly way? I got you.

What happens if I make those branches, but dont commit?

If you create branches but **don't commit** your changes, the branches will still exist, but they won't have any content saved in them. Here's what happens in different scenarios:

1. Branching without committing any changes:

- **Branch Creation**: When you create a new branch using `git checkout -b branch-name`, Git simply points to the same commit as your current branch at the time. It's like taking a snapshot of your code at that moment.
- **No Commits Yet**: If you don't make any changes or commits, the branch won't have any unique history or content compared to the branch you branched off from. It's essentially a "clone" of the original branch at the time of creation.

2. What happens when you switch branches?

- **Uncommitted Changes**: If you have changes that are not committed and switch branches, Git will try to move those changes with you (assuming no conflicts between the branches). If there are conflicts (like changes in the same file), Git will prevent the switch and ask you to either commit or stash your changes first.
- **No Changes**: If there are no uncommitted changes and you switch, you can switch between branches freely. However, the new branch will just "point" to the same commit until you make changes and commit them.

3. What happens if you don't commit in a branch?

- **Working directory**: Your working directory will be the same as the last commit in the branch you created from. No new commits means no changes in history.

- **Potential Issues**: If you accidentally forget to commit your changes, they could be lost if you switch branches or if there's any accidental `git reset` or other commands that modify history. That's why it's a good idea to **always commit** your work regularly.

4. Can I push a branch without commits?

You technically **can't push an empty branch** to a remote repository without commits. Git won't allow you to push a branch that doesn't contain any changes. You'll need to make at least one commit before pushing.

💡 Key Takeaways:

- **Branching** without committing creates an empty branch with no new changes.
- **Switching branches** is safe as long as you commit or stash your changes before switching. Otherwise, Git might stop you if there are conflicts.
- **Empty branches** can't be pushed to a remote repository until there is at least one commit.

📝 Best Practice:

> **Commit regularly** to keep track of changes, even if it's just small adjustments. This way, you avoid losing progress and ensure you have something meaningful in each branch.

Let me know if that clears things up or if you need more guidance!